

SRM 供应商管理系统 底座功能契约文档

(登录 / 权限拦截 / 统一异常处理 / 审计与日志 · v1)

模块 1: 登录

1.1 对外接口

POST /api/v1/auth/login

唯一无需携带 token 的接口。

Request:

```
{ "username": "staff01", "password": "*****" }
```

Response data:

```
{
  "token": "eyJhbGciOi...",
  "user_id": 2,
  "username": "staff01",
  "real_name": "张三",
  "role": "STAFF"
}
```

TypeScript 类型定义:

```
interface LoginRequest { username: string; password: string; }
interface LoginResponse {
  token: string;
  user_id: number;
  username: string;
  real_name: string;
  role: 'ADMIN' | 'STAFF' | 'AUDITOR';
}
```

MVP 演示账号 (启动时按需播种, 仅当 users 表为空, 受 srm.seed-demo-users 配置开关控制) :

账号	密码	角色
admin	Admin@123	ADMIN
staff01	Staff@123	STAFF
auditor01	Auditor@123	AUDITOR

1.2 内部实现：JWT 鉴权组件

登录成功后由 `JwtTokenProvider` 签发 token；后续请求携带 `Authorization: Bearer <token>`，由 `JwtAuthenticationFilter` 在进入 Controller 前完成校验并写入 `SecurityContext`。

```
@Component
class JwtTokenProvider {
    String generateToken(UserPrincipal principal);
    boolean validateToken(String token);          // 过期/签名不合法均返回 false，不
    抛异常
    String getUsername(String token);
    Role getRole(String token);
}

class JwtAuthenticationFilter extends OncePerRequestFilter {
    // 从 Authorization: Bearer <token> 取 token，校验通过则塞入
    SecurityContext;
    // 校验失败不抛异常，直接放行给后续的 RestAuthenticationEntryPoint 统一处理为
    40101
}

@Component
class CustomUserDetailsService implements UserDetailsService {
    UserDetails loadUserByUsername(String username); // 找不到/enabled=false 均
    抛 UsernameNotFoundException
}

class UserPrincipal implements UserDetails {
    Long getId();
    Role getRole();
    // 其余为 UserDetails 标准方法
}
```

JwtAuthenticationFilter 校验失败时不抛异常、只放行——真正的“未登录/token 失效”响应统一交给模块 3 的 RestAuthenticationEntryPoint 处理，避免鉴权逻辑和异常响应

模块 2：权限拦截（RBAC）

2.1 前后端必须双重校验

三大角色：ADMIN（系统管理员）、STAFF（业务员）、AUDITOR（审核员）。ADMIN 原则上不参与供应商业务数据的新增/修改/审批，管理权限与业务权限严格分离。

2.2 RBAC 权限矩阵

业务操作	允许角色	限制条件
供应商信息新增/编辑	STAFF	仅限【草稿】/【退回】状态； 【待审核】及以后状态后端强制 拒绝写操作
资质审核（通过/驳回）	AUDITOR	—
绩效复核	AUDITOR	—
停用/淘汰最终决策	AUDITOR	—
停用/恢复/淘汰申请	STAFF 提出，AUDITOR 决策	两步流程（LifecycleRequest）
用户/角色管理	ADMIN	—
删除供应商草稿	STAFF（本人）	仅限本人草稿/待提交状态；正 式流程记录禁止物理删除，只能 逻辑删除

2.3 Controller 层实现方式

角色校验用 `@PreAuthorize("hasRole(...)")` 声明，示例：

```
@PreAuthorize("hasRole('STAFF')")
ApiResponse<SupplierResponse> createSupplier(...);

@PreAuthorize("hasRole('AUDITOR')")
ApiResponse<SupplierResponse> auditSupplier(...);

@PreAuthorize("hasRole('ADMIN')")
ApiResponse<UserResponse> createUser(...);
```

所有方法返回 `ApiResponse<T>`；分页接口返回 `ApiResponse<PageResult<T>>`。 `@PreAuthorize` 只做角色粗粒度拦截，“本人”/“仅限某状态”这类细粒度必须在 `Service` 层二次校验。

2.4 拦截失败的响应

```
@Component
class RestAccessDeniedHandler implements AccessDeniedHandler {
    // 角色不匹配 → 返回 ApiResponse.error(40301, "无权限")
}
```

40301 的完整语义与其他错误码见模块 3「统一异常处理」。

模块 3：统一异常处理

对应 docs/api-spec.md 第 8 节（错误码表）与 docs/backend-interface-design.md 第 9 节（Common 组件接口）。

3.1 统一响应体

```
interface ApiResponse<T> {
    code: number;           // 0 = 成功; 非 0 见下方错误码表
    message: string;
    data: T | null;
}
```

分页响应统一为 **PageResult<T>**：

```
interface PageResult<T> {
    list: T[];
    total: number;
    page: number;
    page_size: number;
}
```

3.2 错误码表

code	含义
0	成功
40001	参数校验失败（含必填缺失、格式错误、事实数据非法）
40101	未登录 / token 失效
40301	无权限（角色不匹配）
40302	当前状态不允许该操作（如非草稿态编辑、非本人操作）
40401	资源不存在
40901	并发冲突（乐观锁版本不匹配）

40902	业务冲突（重复的在途申请/评价单）
50001	服务器内部错误

3.3 内部实现：Common 组件接口

```

public class ApiResponse<T> {
    int code; String message; T data;
    static <T> ApiResponse<T> success(T data);
    static <T> ApiResponse<T> error(ErrorCode code, String message);
}

public class PageResult<T> {
    List<T> list; long total; int page; int pageSize;
}

public class BusinessException extends RuntimeException {
    BusinessException(ErrorCode code, String message);
    ErrorCode getCode();
}

public enum ErrorCode {
    SUCCESS(0), VALIDATION_FAILED(40001), UNAUTHORIZED(40101),
    FORBIDDEN(40301),
    STATUS_NOT_ALLOWED(40302), NOT_FOUND(40401),
    OPTIMISTIC_LOCK_CONFLICT(40901),
    DUPLICATE_IN_PROGRESS(40902), INTERNAL_ERROR(50001);
    final int value;
}

```

3.4 全局异常处理器

```

@RestControllerAdvice
class GlobalExceptionHandler {
    @ExceptionHandler(BusinessException.class)
    ApiResponse<Void> handleBusiness(BusinessException ex);

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ApiResponse<Void> handleValidation(MethodArgumentNotValidException ex); //
    → 40001

    @ExceptionHandler(OptimisticLockingFailureException.class)
    ApiResponse<Void> handleOptimisticLock(OptimisticLockingFailureException
    ex); // → 40901

    @ExceptionHandler(Exception.class)

```

```

    ApiResponse<Void> handleUnknown(Exception ex); // → 50001, 同时记 ERROR 日志
}

@Component
class RestAuthenticationEntryPoint implements AuthenticationEntryPoint {
    // 未登录/token 失效 → 返回 ApiResponse.error(40101, "未登录或登录已过期")
}

```

40101/40301 由 Security 层的 RestAuthenticationEntryPoint / RestAccessDeniedHandler 处理 (见模块 1、2) , 不经过 GlobalExceptionHandler; 其余业务异常统一走 GlobalExceptionHandler, 保证响应体格式唯一。

模块 4: 审计与日志

4.1 对外接口

```

interface AuditLogResponse {
    id: number;
    entity_type: 'SUPPLIER' | 'PERFORMANCE_EVALUATION' |
'PERFORMANCE_EVALUATION_DRAFT' | 'LIFECYCLE_REQUEST';
    entity_id: number;
    operator_id: number;
    operator_name: string;
    operator_role: Role;
    action: string;
    old_status?: string;
    new_status?: string;
    result: 'SUCCESS' | 'REJECTED';
    comment?: string;
    created_at: string;
}

```

方法	路径	角色	说明
GET	/audit-logs	ALL	query: entity_type=&entity_id=&page=&page_size=; 前端按"业务范围内"过滤展示, 后端不做行级隔离

4.2 action 完整枚举

action	含义
SUBMIT	供应商 / 绩效评价草稿提交审核
AUDIT_APPROVE	供应商资质审核通过

AUDIT_REJECT	供应商资质审核驳回
REVIEW_APPROVE	绩效评价复核通过
REVIEW_REJECT	绩效评价复核驳回
DECISION_APPROVE	生命周期申请（停用/恢复/淘汰）决策通过
DECISION_REJECT	生命周期申请（停用/恢复/淘汰）决策驳回

entity_type = LIFECYCLE_REQUEST 时, DECISION_APPROVE/DECISION_REJECT 统一表示停用/恢复/淘汰三种申请类型的决策, 具体申请类型看关联的 lifecycle_request.type, 不再按类型拆分 action 值。

4.3 内部实现: AuditLogService

```
public interface AuditLogService {
    void record(EntityType entityType, Long entityId, UserPrincipal operator,
                String action, SupplierStatus oldStatus, SupplierStatus
newStatus,
                AuditResult result, String comment);
    PageResult<AuditLogResponse> list(String entityType, Long entityId, int
page, int pageSize);
}
```

record(...) 内部的 Mapper 写入失败不得向上抛出、中断主业务事务——记日志失败只应打 ERROR 日志, 不应导致审核/提交等主操作跟着回滚失败 (已有测试 AuditLogServiceImplTest#recordNeverPropagatesMapperFailures 覆盖这一约束)。

4.4 必须写审计日志的场景

- 供应商审核决策 (POST /suppliers/{id}/audit, APPROVE/REJECT)
- 生命周期申请决策 (POST /lifecycle-requests/{id}/decision, APPROVE/REJECT)
- 绩效评价复核 (POST /performance/evaluations/{id}/review, APPROVE/REJECT)
- 并发冲突 (40901) 或业务冲突 (40902) 导致操作失败时, 也应写入一条 result=REJECTED 的记录, 不能静默失败不留痕